

# CUDA - lab3

Jakub Ciszewski

## Programy

```
!pip install nvcc4jupyter
%load_ext nvcc4jupyter

Collecting nvcc4jupyter
  Downloading nvcc4jupyter-1.2.1-py3-none-any.whl.metadata (5.1 kB)
Downloading nvcc4jupyter-1.2.1-py3-none-any.whl (10 kB)
Installing collected packages: nvcc4jupyter
Successfully installed nvcc4jupyter-1.2.1
Detected platform "Colab". Running its setup...
Source files will be saved in "/tmp/tmp7gq85vk5".
```

```
%%writefile kernel.h
```

```
#ifndef KERNEL_CUH_
#define KERNEL_CUH_
```

```
void matrixMultiplication(float *A, float *B, float *C, int N);
```

```
#endif
```

```
Overwriting kernel.h
```

```
%%writefile devarray.h
```

```
#ifndef _DEV_ARRAY_H_
#define _DEV_ARRAY_H_
```

```
#include <stdexcept>
#include <algorithm>
#include <cuda_runtime.h>
```

```
template <class T>
class dev_array
{
// public functions
public:
    explicit dev_array()
        : start_(0),
          end_(0)
    {}

    // constructor
    explicit dev_array(size_t size)
```

```

{
    allocate(size);
}
// destructor
~dev_array()
{
    free();
}

// resize the vector
void resize(size_t size)
{
    free();
    allocate(size);
}

// get the size of the array
size_t getSize() const
{
    return end_ - start_;
}

// get data
const T* getData() const
{
    return start_;
}

T* getData()
{
    return start_;
}

// set
void set(const T* src, size_t size)
{
    size_t min = std::min(size, getSize());
    cudaError_t result = cudaMemcpy(start_, src, min * sizeof(T),
cudaMemcpyHostToDevice);
    if (result != cudaSuccess)
    {
        throw std::runtime_error("failed to copy to device
memory");
    }
}

// get
void get(T* dest, size_t size)
{
    size_t min = std::min(size, getSize());
    cudaError_t result = cudaMemcpy(dest, start_, min * sizeof(T),

```

```

cudaMemcpyDeviceToHost);
    if (result != cudaSuccess)
    {
        throw std::runtime_error("failed to copy to host memory");
    }
}

// private functions
private:
    // allocate memory on the device
    void allocate(size_t size)
    {
        cudaError_t result = cudaMalloc((void**)&start_, size *
sizeof(T));
        if (result != cudaSuccess)
        {
            start_ = end_ = 0;
            throw std::runtime_error("failed to allocate device
memory");
        }
        end_ = start_ + size;
    }

    // free memory on the device
    void free()
    {
        if (start_ != 0)
        {
            cudaFree(start_);
            start_ = end_ = 0;
        }
    }

    T* start_;
    T* end_;
};

#endif

```

Overwriting devarray.h

```
%%writefile kernel.cu
```

```

#include <math.h>
#include <iostream>
#include "cuda_runtime.h"
#include "kernel.h"
#include <stdlib.h>

```

```

using namespace std;

__global__ void matrixMultiplicationKernel(float* A, float* B, float*
C, int N) {

    int ROW = blockIdx.y*blockDim.y+threadIdx.y;
    int COL = blockIdx.x*blockDim.x+threadIdx.x;

    float tmpSum = 0;

    if (ROW < N && COL < N) {
        // each thread computes one element of the block sub-matrix
        for (int i = 0; i < N; i++) {
            tmpSum += A[ROW * N + i] * B[i * N + COL];
        }
        C[ROW * N + COL] = tmpSum;
    }

}

void matrixMultiplication(float *A, float *B, float *C, int N){

    // declare the number of blocks per grid and the number of threads
    per block
    // use 1 to 512 threads per block
    dim3 threadsPerBlock(N, N);
    dim3 blocksPerGrid(1, 1);
    if (N*N > 512){
        threadsPerBlock.x = 512;
        threadsPerBlock.y = 512;
        blocksPerGrid.x =
ceil(double(N)/double(threadsPerBlock.x));
        blocksPerGrid.y =
ceil(double(N)/double(threadsPerBlock.y));
    }

    matrixMultiplicationKernel<<<blocksPerGrid,threadsPerBlock>>>(A,
B, C, N);
}

```

Overwriting kernel.cu

```

%%shell
nvcc kernel.cu -o kernel -arch=sm_75 -lcublas

/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/11/../../../../x86_64-linux-
gnu/Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status

```

```

-----
-----
CalledProcessError                                Traceback (most recent call
last)
/tmp/ipykernel_622/2475387040.py in <cell line: 0>()
----> 1 get_ipython().run_cell_magic('shell', '', 'nvcc kernel.cu -o
kernel -arch=sm_75 -lcublas\n')

/usr/local/lib/python3.12/dist-packages/google/colab/_shell.py in
run_cell_magic(self, magic_name, line, cell)
    221     if line and not cell:
    222         cell = ' '
--> 223     return super().run_cell_magic(magic_name, line, cell)
    224
    225

/usr/local/lib/python3.12/dist-packages/IPython/core/interactiveshell.
py in run_cell_magic(self, magic_name, line, cell)
    2471         with self.builtin_trap:
    2472             args = (magic_arg_s, cell)
-> 2473             result = fn(*args, **kwargs)
    2474         return result
    2475

/usr/local/lib/python3.12/dist-packages/google/colab/_system_commands.
py in _shell_cell_magic(args, cmd)
    110     result = _run_command(cmd, clear_streamed_output=False)
    111     if not parsed_args.ignore_errors:
--> 112     result.check_returncode()
    113     return result
    114

/usr/local/lib/python3.12/dist-packages/google/colab/_system_commands.
py in check_returncode(self)
    135     def check_returncode(self):
    136         if self.returncode:
--> 137             raise subprocess.CalledProcessError(
    138                 returncode=self.returncode, cmd=self.args,
output=self.output
    139             )

CalledProcessError: Command 'nvcc kernel.cu -o kernel -arch=sm_75 -
lcublas
' returned non-zero exit status 1.

%%writefile matrixmul_cublas_speed.cu

#include <iostream>
#include <chrono>
#include <cublas_v2.h>

```

```

#include <cstdlib> // Potrzebne do std::atoi

// Macro for checking cuBLAS errors
#define CHECK_CUBLAS(call) \
    do { \
        cublasStatus_t status = (call); \
        if (status != CUBLAS_STATUS_SUCCESS) { \
            std::cerr << "CUBLAS Error: " << status << " in " << #call \
<< " at line " << __LINE__ << std::endl; \
            exit(1); \
        } \
    } while (0)

// Macro for checking CUDA errors
#define CHECK_CUDA(call) \
    do { \
        cudaError_t status = (call); \
        if (status != cudaSuccess) { \
            std::cerr << "CUDA Error: " << cudaGetErrorString(status) \
<< " in " << #call << " at line " << __LINE__ << std::endl; \
            exit(1); \
        } \
    } while (0)

#define IDX2C(i, j, ld) (((j)*(ld))+(i)) // Column-major indexing

int main(int argc, char* argv[]) {
    int size = 1024; // Domyślny rozmiar macierzy (N x N)

    // Odczyt argumentu z linii komend
    if (argc > 1) {
        size = std::atoi(argv[1]);
        if (size <= 0) {
            std::cerr << "Błąd: Rozmiar macierzy musi być liczba
większa od 0!" << std::endl;
            return 1;
        }
    } else {
        std::cout << "Nie podano argumentu. Używam domyślnego
rozmiaru: " << size << "x" << size << std::endl;
    }

    std::cout << "Uruchamianie testu wydajności cuBLAS dla rozmiaru: "
<< size << "x" << size << std::endl;

    // Allocate host memory
    float *h_A, *h_B, *h_C_host;
    h_A = new float[size * size];
    h_B = new float[size * size];
    h_C_host = new float[size * size];

```

```

// Initialize host matrices (column-major for cuBLAS)
for (int j = 0; j < size; ++j) {
    for (int i = 0; i < size; ++i) {
        h_A[IDX2C(i, j, size)] = 1.0f;
        h_B[IDX2C(i, j, size)] = 2.0f;
    }
}

// Allocate device memory
float *d_A, *d_B, *d_C_device;
CHECK_CUDA(cudaMalloc(&d_A, size * size * sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_B, size * size * sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_C_device, size * size * sizeof(float)));

// Copy data from host to device
CHECK_CUDA(cudaMemcpy(d_A, h_A, size * size * sizeof(float),
    cudaMemcpyHostToDevice));
CHECK_CUDA(cudaMemcpy(d_B, h_B, size * size * sizeof(float),
    cudaMemcpyHostToDevice));

// Initialize cuBLAS
cublasHandle_t cublasHandle;
CHECK_CUBLAS(cublasCreate(&cublasHandle));

// Perform matrix multiplication using cuBLAS ( $C = \alpha * A * B + \beta * C$ )
// For  $C = A * B$ ,  $\alpha = 1.0$ ,  $\beta = 0.0$ 
float alpha = 1.0f;
float beta = 0.0f;

// Matrix dimensions (N x N)
int n = size;

// Warm-up call (optional, but good practice for timing)
CHECK_CUBLAS(cublasSgemv(cublasHandle,
    CUBLAS_OP_N,
    CUBLAS_OP_N,
    n, n, n,
    &alpha,
    d_A, n,
    d_B, n,
    &beta,
    d_C_device, n));
CHECK_CUDA(cudaDeviceSynchronize());

// Measure cuBLAS execution time
auto start = std::chrono::high_resolution_clock::now();
CHECK_CUBLAS(cublasSgemv(cublasHandle,
    CUBLAS_OP_N,

```

```

        CUBLAS_OP_N,
        n,
        n,
        n,
        &alpha,
        d_A,
        n,
        d_B,
        n,
        &beta,
        d_C_device,
        n));

CHECK_CUDA(cudaDeviceSynchronize()); // Wait for the cuBLAS call
to complete
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate duration
    std::chrono::duration<double> duration = end - start;

    // Copy result from device to host
    CHECK_CUDA(cudaMemcpy(h_C_host, d_C_device, size * size *
sizeof(float), cudaMemcpyDeviceToHost));

    // Verify result
    // For h_A = 1.0 and h_B = 2.0, each element of C should be size *
1.0 * 2.0
    float expected = static_cast<float>(size) * 1.0f * 2.0f;
    // Check a few elements to verify
    bool error_found = false;
    if (std::abs(h_C_host[IDX2C(0, 0, size)] - expected) > 1e-3) { //
Use a tolerance for float comparison
        std::cerr << "Error: Expected " << expected << ", Got " <<
h_C_host[IDX2C(0, 0, size)] << " Difference: " << h_C_host[IDX2C(0, 0,
size)] - expected << std::endl;
        error_found = true;
    }
    if (std::abs(h_C_host[IDX2C(size-1, size-1, size)] - expected) >
1e-3) { // Use a tolerance for float comparison
        std::cerr << "Error: Expected " << expected << ", Got " <<
h_C_host[IDX2C(size-1, size-1, size)] << " Difference: " <<
h_C_host[IDX2C(size-1, size-1, size)] - expected << std::endl;
        error_found = true;
    }

    if (!error_found) {
        std::cout << "Result verification successful (checked sample
elements)." << std::endl;
    }

```

```
std::cout << "cuBLAS execution time (with warm-up): " <<
duration.count() * 1000 << " ms" << std::endl;
```

```
// Free cuBLAS resources
CHECK_CUBLAS(cublasDestroy(cublasHandle));
```

```
// Free device memory
CHECK_CUDA(cudaFree(d_A));
CHECK_CUDA(cudaFree(d_B));
CHECK_CUDA(cudaFree(d_C_device));
```

```
// Free host memory
delete[] h_A;
delete[] h_B;
delete[] h_C_host;
```

```
return 0;
```

```
}
```

Overwriting matrixmul\_cublas\_speed.cu

```
%%shell
```

```
nvcc matrixmul_cublas_speed.cu -o matrixmul_cublas_speed -arch=sm_75 -lcublas
```

```
%%writefile matrixmul_cublas.cu
```

```
#include <iostream>
```

```
#include <chrono>
```

```
#include <cublas_v2.h>
```

```
#include <cstdlib> // Potrzebne do std::atoi
```

```
#define IDX2C(i, j, ld) (((j)*(ld))+(i)) // Column-major indexing
```

```
int main(int argc, char* argv[]) {
    int size = 1024; // Domyślny rozmiar macierzy (N x N)
```

```
    // Odczyt argumentu z linii komend
```

```
    if (argc > 1) {
        size = std::atoi(argv[1]);
```

```
        if (size <= 0) {
            std::cerr << "Błąd: Rozmiar macierzy musi być liczba
większa od 0!" << std::endl;
            return 1;
        }
    }
}
```

```
    } else {
        std::cout << "Nie podano argumentu. Używam domyślnego
rozmiaru: " << size << "x" << size << std::endl;
```

```

    }

    std::cout << "Uruchamianie mnozenia cuBLAS dla rozmiaru: " << size
    << "x" << size << std::endl;

    // Allocate host memory
    float *h_A, *h_B, *h_C_host;
    h_A = new float[size * size];
    h_B = new float[size * size];
    h_C_host = new float[size * size];

    // Initialize host matrices (column-major for cuBLAS)
    for (int j = 0; j < size; ++j) {
        for (int i = 0; i < size; ++i) {
            h_A[IDX2C(i, j, size)] = 1.0f;
            h_B[IDX2C(i, j, size)] = 2.0f;
        }
    }

    // Allocate device memory
    float *d_A, *d_B, *d_C_device;
    cudaMalloc(&d_A, size * size * sizeof(float));
    cudaMalloc(&d_B, size * size * sizeof(float));
    cudaMalloc(&d_C_device, size * size * sizeof(float));

    // Copy data from host to device
    cudaMemcpy(d_A, h_A, size * size * sizeof(float),
    cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size * size * sizeof(float),
    cudaMemcpyHostToDevice);

    // Initialize cuBLAS
    cublasHandle_t cublasHandle;
    cublasCreate(&cublasHandle);

    // Perform matrix multiplication using cuBLAS ( $C = \alpha * A * B + \beta * C$ )
    // For  $C = A * B$ ,  $\alpha = 1.0$ ,  $\beta = 0.0$ 
    float alpha = 1.0f;
    float beta = 0.0f;

    // Matrix dimensions (N x N)
    int n = size;

    // Measure cuBLAS execution time
    auto start = std::chrono::high_resolution_clock::now();
    // cublasSgemm(handle, transa, transb, m, n, k, alpha, A, lda, B,
    ldb, beta, C, ldc)
    // For  $C = A * B$ , where A, B, C are N x N matrices
    // m = N, n = N, k = N

```

```

// lda = N, ldb = N, ldc = N
// A and B are in column-major order.
cublasSgemm(cublasHandle,
            CUBLAS_OP_N, // A is not transposed
            CUBLAS_OP_N, // B is not transposed
            n,           // m: number of rows of A and C
            n,           // n: number of columns of B and C
            n,           // k: number of columns of A and rows of
B
            &alpha,      // scalar alpha
            d_A,         // A matrix on device
            n,           // lda: leading dimension of A
            d_B,         // B matrix on device
            n,           // ldb: leading dimension of B
            &beta,       // scalar beta
            d_C_device,  // C matrix on device
            n);          // ldc: leading dimension of C

cudaDeviceSynchronize(); // Wait for the cuBLAS call to complete
auto end = std::chrono::high_resolution_clock::now();

// Calculate duration
std::chrono::duration<double> duration = end - start;

// Copy result from device to host
cudaMemcpy(h_C_host, d_C_device, size * size * sizeof(float),
cudaMemcpyDeviceToHost);

// Verify result
// For h_A = 1.0 and h_B = 2.0, each element of C should be size *
1.0 * 2.0
float expected = static_cast<float>(size) * 1.0f * 2.0f;
// Check a few elements to verify
bool error_found = false;
if (h_C_host[IDX2C(0, 0, size)] != expected) {
    std::cerr << "Error: Expected " << expected << ", Got " <<
h_C_host[IDX2C(0, 0, size)] << " Difference: " << h_C_host[IDX2C(0, 0,
size)] - expected << std::endl;
    error_found = true;
}
if (h_C_host[IDX2C(size-1, size-1, size)] != expected) {
    std::cerr << "Error: Expected " << expected << ", Got " <<
h_C_host[IDX2C(size-1, size-1, size)] << " Difference: " <<
h_C_host[IDX2C(size-1, size-1, size)] - expected << std::endl;
    error_found = true;
}

if (!error_found) {
    std::cout << "Result verification successful (checked sample
elements)." << std::endl;
}

```

```

    }

    std::cout << "cuBLAS execution time: " << duration.count() * 1000
    << " ms" << std::endl;

    // Free cuBLAS resources
    cublasDestroy(cublasHandle);

    // Free device memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C_device);

    // Free host memory
    delete[] h_A;
    delete[] h_B;
    delete[] h_C_host;

    return 0;
}

Overwriting matrixmul_cublas.cu

%%shell

nvcc matrixmul_cublas.cu -o matrixmul_cublas -arch=sm_75 -lcublas

%%writefile matrixmul_shared.cu

#include <iostream>
#include <chrono>
#include <cstdlib> // Potrzebne do std::atoi

// CUDA kernel for matrix multiplication using shared memory
__global__ void matrixMul(float *A, float *B, float *C, int size) {
    // Thread index within block
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Global row and column index
    int row = blockIdx.y * blockDim.y + ty;
    int col = blockIdx.x * blockDim.x + tx;

    // Define shared memory for block A and B
    // Block size must be known at compile time for shared memory
allocation
    // For this example, we assume blockDim.x and blockDim.y are equal
to `TILE_DIM`
    extern __shared__ float s_A[]; // TILE_DIM * TILE_DIM

```

```

extern __shared__ float s_B[]; // TILE_DIM * TILE_DIM

float Cvalue = 0;

// Loop over the tiles of the input matrices
// 'size' is the dimension of the square matrices
int TILE_DIM = blockDim.x; // Assuming square blocks

for (int m = 0; m < (size + TILE_DIM - 1) / TILE_DIM; ++m) {
    // Load a tile of A and B into shared memory
    int aRow = row;
    int aCol = m * TILE_DIM + tx;
    int bRow = m * TILE_DIM + ty;
    int bCol = col;

    // Boundary check for loading into shared memory
    if (aRow < size && aCol < size) {
        s_A[ty * TILE_DIM + tx] = A[aRow * size + aCol];
    } else {
        s_A[ty * TILE_DIM + tx] = 0.0f; // Pad with zeros if out
of bounds
    }

    if (bRow < size && bCol < size) {
        s_B[ty * TILE_DIM + tx] = B[bRow * size + bCol]; // Pad
with zeros if out of bounds
    } else {
        s_B[ty * TILE_DIM + tx] = 0.0f; // Pad with zeros if out
of bounds
    }

    __syncthreads(); // Synchronize all threads in the block to
ensure tile is loaded

    // Perform the matrix multiplication for the current tiles
    for (int k = 0; k < TILE_DIM; ++k) {
        Cvalue += s_A[ty * TILE_DIM + k] * s_B[k * TILE_DIM + tx];
    }

    __syncthreads(); // Synchronize again to ensure shared memory
is flushed before next tile load
}

// Store the final result in global memory
if (row < size && col < size) {
    C[row * size + col] = Cvalue;
}
}

int main(int argc, char* argv[]) {

```

```

int size = 1024; // Domyślny rozmiar macierzy

// Odczyt argumentu z linii komend
if (argc > 1) {
    size = std::atoi(argv[1]);
    if (size <= 0) {
        std::cerr << "Błąd: Rozmiar macierzy musi być liczba
większa od 0!" << std::endl;
        return 1;
    }
} else {
    std::cout << "Nie podano argumentu. Używam domyślnego
rozmiaru: " << size << "x" << size << std::endl;
}

std::cout << "Uruchamianie mnożenia dla rozmiaru: " << size << "x"
<< size << std::endl;

// Allocate host memory
float *h_A, *h_B, *h_C;
h_A = new float[size * size];
h_B = new float[size * size];
h_C = new float[size * size];

// Initialize host matrices
for (int i = 0; i < size * size; ++i) {
    h_A[i] = 1.0f;
    h_B[i] = 2.0f;
}

// Allocate device memory
float *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size * size * sizeof(float));
cudaMalloc(&d_B, size * size * sizeof(float));
cudaMalloc(&d_C, size * size * sizeof(float));

// Copy data from host to device
cudaMemcpy(d_A, h_A, size * size * sizeof(float),
cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size * size * sizeof(float),
cudaMemcpyHostToDevice);

// Define grid and block dimensions
int blockSize = 32; // Changed block size for shared memory
optimization
dim3 blockDim(blockSize, blockSize);
dim3 gridDim((size + blockSize - 1) / blockSize, (size + blockSize
- 1) / blockSize);

// Calculate shared memory size needed for the kernel

```

```

    // Each shared memory array holds TILE_DIM * TILE_DIM floats.
    // Since we have two (s_A and s_B), it's 2 * TILE_DIM * TILE_DIM *
sizeof(float).
    int sharedMemSize = 2 * blockSize * blockSize * sizeof(float);

    // Measure kernel execution time
    auto start = std::chrono::high_resolution_clock::now();
    matrixMul<<<gridDim, blockDim, sharedMemSize>>>(d_A, d_B, d_C,
size);
    cudaDeviceSynchronize(); // Wait for the kernel to finish
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate duration
    std::chrono::duration<double> duration = end - start;

    // Copy result from device to host
    cudaMemcpy(h_C, d_C, size * size * sizeof(float),
cudaMemcpyDeviceToHost);

    // Verify result (optional, for debugging)
    // For this example, with h_A = 1.0 and h_B = 2.0, each element of
C should be size * 1.0 * 2.0
    //float expected = static_cast<float>(size) * 1.0f * 2.0f;
    //if (h_C[0] != expected) {
    //    std::cerr << "Error: Expected " << expected << ", Got " <<
h_C[0] << " Difference: " << h_C[0] - expected << std::endl;
    //}

    std::cout << "Kernel execution time (with shared memory): " <<
duration.count() * 1000 << " ms" << std::endl;

    // Free device memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    // Free host memory
    delete[] h_A;
    delete[] h_B;
    delete[] h_C;

    return 0;
}

```

Overwriting matrixmul\_shared.cu

%%shell

nvcc matrixmul\_shared.cu -o matrixmul\_shared -arch=sm\_75 -lcublas

```

%%writefile matrixmul.cu

#include <iostream>
#include <chrono>
#include <cstdlib> // Potrzebne do std::atoi

// CUDA kernel for matrix multiplication
__global__ void matrixMul(float *A, float *B, float *C, int size) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < size && col < size) {
        float sum = 0.0f;
        for (int i = 0; i < size; ++i) {
            sum += A[row * size + i] * B[i * size + col];
        }
        C[row * size + col] = sum;
    }
}

int main(int argc, char* argv[]) {
    // Domyślny rozmiar, jeśli użytkownik nic nie poda
    int size = 1024;

    // Sprawdzenie, czy użytkownik podał argument
    if (argc > 1) {
        size = std::atoi(argv[1]);
        if (size <= 0) {
            std::cerr << "Błąd: Rozmiar macierzy musi być liczbą
większą od 0!" << std::endl;
            return 1;
        }
    } else {
        std::cout << "Nie podano argumentu. Używam domyślnego
rozmiaru: " << size << "x" << size << std::endl;
    }

    std::cout << "Uruchamianie mnożenia macierzy o rozmiarze: " <<
size << "x" << size << std::endl;

    // Alokacja pamięci na hoście (używamy size_t dla bezpieczeństwa
przy dużych macierzach)
    size_t num_elements = (size_t)size * size;
    float *h_A = new float[num_elements];
    float *h_B = new float[num_elements];
    float *h_C = new float[num_elements];

    // Initialize host matrices
    for (size_t i = 0; i < num_elements; ++i) {
        h_A[i] = 1.0f;
    }
}

```

```

        h_B[i] = 2.0f;
    }

    // Allocate device memory
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, num_elements * sizeof(float));
    cudaMalloc(&d_B, num_elements * sizeof(float));
    cudaMalloc(&d_C, num_elements * sizeof(float));

    // Copy data from host to device
    cudaMemcpy(d_A, h_A, num_elements * sizeof(float),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, num_elements * sizeof(float),
cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 16;
    dim3 blockDim(blockSize, blockSize);
    dim3 gridDim((size + blockSize - 1) / blockSize, (size + blockSize
- 1) / blockSize);

    // Measure kernel execution time
    auto start = std::chrono::high_resolution_clock::now();
    matrixMul<<<gridDim, blockDim>>>(d_A, d_B, d_C, size);
    cudaDeviceSynchronize(); // Wait for the kernel to finish
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate duration
    std::chrono::duration<double> duration = end - start;

    // Copy result from device to host
    cudaMemcpy(h_C, d_C, num_elements * sizeof(float),
cudaMemcpyDeviceToHost);

    std::cout << "Kernel execution time: " << duration.count() * 1000
<< " ms" << std::endl;

    // Free device memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    // Free host memory
    delete[] h_A;
    delete[] h_B;
    delete[] h_C;

    return 0;
}

```

```
Overwriting matrixmul.cu
```

```
%%shell
```

```
nvcc matrixmul.cu -o matrixmul -arch=sm_75 -lcublas
```

## Wywołania

```
%%shell
```

```
./matrixmul 1024
```

```
./matrixmul 2048
```

```
./matrixmul 4096
```

```
Uruchamianie mnożenia macierzy o rozmiarze: 1024x1024
```

```
Kernel execution time: 5.7318 ms
```

```
Uruchamianie mnożenia macierzy o rozmiarze: 2048x2048
```

```
Kernel execution time: 39.5923 ms
```

```
Uruchamianie mnożenia macierzy o rozmiarze: 4096x4096
```

```
Kernel execution time: 301.175 ms
```

```
%%shell
```

```
./matrixmul_shared 1024
```

```
./matrixmul_shared 2048
```

```
./matrixmul_shared 4096
```

```
Uruchamianie mnożenia dla rozmiaru: 1024x1024
```

```
Kernel execution time (with shared memory): 6.89559 ms
```

```
Uruchamianie mnożenia dla rozmiaru: 2048x2048
```

```
Kernel execution time (with shared memory): 34.3643 ms
```

```
Uruchamianie mnożenia dla rozmiaru: 4096x4096
```

```
Kernel execution time (with shared memory): 221.489 ms
```

```
%%shell
```

```
./matrixmul_cublas 1024
```

```
./matrixmul_cublas 2048
```

```
./matrixmul_cublas 4096
```

```
Uruchamianie mnożenia cuBLAS dla rozmiaru: 1024x1024
```

```
Result verification successful (checked sample elements).
```

```
cuBLAS execution time: 6.15006 ms
```

```
Uruchamianie mnożenia cuBLAS dla rozmiaru: 2048x2048
```

```
Result verification successful (checked sample elements).
```

```
cuBLAS execution time: 8.83584 ms
```

```
Uruchamianie mnożenia cuBLAS dla rozmiaru: 4096x4096
```

```
Result verification successful (checked sample elements).  
cuBLAS execution time: 30.1618 ms
```

```
%%shell  
./matrixmul_cublas_speed 1024  
./matrixmul_cublas_speed 2048  
./matrixmul_cublas_speed 4096
```

```
Uruchamianie testu wydajnosci cuBLAS dla rozmiaru: 1024x1024  
Result verification successful (checked sample elements).  
cuBLAS execution time (with warm-up): 0.738892 ms  
Uruchamianie testu wydajnosci cuBLAS dla rozmiaru: 2048x2048  
Result verification successful (checked sample elements).  
cuBLAS execution time (with warm-up): 3.3124 ms  
Uruchamianie testu wydajnosci cuBLAS dla rozmiaru: 4096x4096  
Result verification successful (checked sample elements).  
cuBLAS execution time (with warm-up): 25.4352 ms
```

## Wyniki

| Rozmiar macierzy   | Zwykły Kernel (matrixmul) | Pamięć współdzielona (shared) | cuBLAS (bez warm-up) | cuBLAS Speed (z warm-up) |
|--------------------|---------------------------|-------------------------------|----------------------|--------------------------|
| <b>1024 x 1024</b> | 5.73 ms                   | 6.90 ms                       | 6.15 ms              | 0.74 ms                  |
| <b>2048 x 2048</b> | 39.59 ms                  | 34.36 ms                      | 8.84 ms              | 3.31 ms                  |
| <b>4096 x 4096</b> | 301.18 ms                 | 221.49 ms                     | 30.16 ms             | 25.44 ms                 |

## Komentarze

matrixmul.cu

1. Czy występuje coalesced access? Dostęp łączony nie występuje z racji na to, że dostęp do macierzy A nie jest związany ze współrzędną X wątku.
2. Dlaczego wyznaczamy gridDim w taki "dziwny" sposób? Wyznaczamy to w ten sposób żeby zaokrąglić liczbę bloków w górę, czyli żeby dla liczby niepodzielnej przez wielkość bloku (32) nadal otrzymać odpowiednią liczbę bloków, czyli np dla size=50 i blockSize=32 normalne dzielenie dwóch intów w cpp dałoby nam 1 - 18 komórek zostałyby bez wątków. Jeśli zrobimy  $(50 + 32 - 1) / 32$  to otrzymamy  $81/32=2$ .
3. Co to dim3? dim3 to specjalny typ danych CUDA stosowany do definiowania wymiarów bloków wątków.

matrixmul\_shared.cu

1. Dlaczego `TILE_DIM` jest równe `blockDim`? Żeby zachować zasadę, że jeden wątek ładuje jeden element. Kwadratowy blok wątków idealnie pokrywa wtedy kwadratowy kafelek.
2. Jaki może być maksymalny rozmiar `blockDim`? Maksymalny rozmiar bloku dla tego środowiska wykonawczego to 1024. Czyli maksymalny rozmiar `blockDim` to 32x32, bo  $32*32=1024$ .
3. Czy w tym kodzie dochodzi do `bank conflict`? Tak występuje podczas odczytu z `s_A`.

`matrixmul_cublas.cu` i `matrixmul_cublas_speed.cu`

1. Krótki komentarz co do różnic. Główną różnicą między wersją `speed` i `nie-speed` jest taka, że wersja `speed` przed rozpoczęciem pomiarów robi `warm-up` GPU.